

Laboratorio 5

Sistemas operativos

12/05/2026

Andres Rodriguez

Objetivos y alcance

El objetivo de este laboratorio es comprender la instalación, administración y uso de diferentes sistemas de bases de datos en entornos Linux, utilizando tecnologías como Docker y Azure Cloud. Se trabajó con bases de datos relacionales y NoSQL, realizando tareas de configuración, gestión de servicios, creación de usuarios, consultas, backups y restauración de datos.

El alcance del laboratorio incluye la administración de una máquina virtual Ubuntu en Azure, el despliegue de contenedores Docker, la implementación de MySQL, MongoDB y Redis, la aplicación de conceptos básicos de seguridad, redes y automatización mediante scripts Bash. Esto permite adquirir experiencia práctica en herramientas y tecnologías utilizadas habitualmente en entornos profesionales de infraestructura y DevOps.

PreLaboratorio

Investigación de Motores de Bases de Datos

MySQL

Es un sistema de gestión de bases de datos relacionales de código abierto, es propiedad de Oracle Corporation, reconocido por su velocidad, confiabilidad y facilidad de uso. Su arquitectura se basa en un modelo cliente-servidor y utiliza almacenamiento modular, destacando motores como InnoDB que garantizan transacciones seguras mediante el cumplimiento de las propiedades ACID. Tiene una comunidad grande de desarrolladores, lo que logra un soporte continuo. Es ideal para el desarrollo de aplicaciones web estándar, plataformas de comercio electrónico y sistemas de gestión de contenido dinámicos como WordPress o Drupal.

PostgreSQL

Es un motor relacional orientado a objetos de código abierto que se enfoca en la extensibilidad, la robustez técnica y el cumplimiento de los estándares SQL. Tiene la capacidad para manejar tipos de datos complejos de forma nativa, tales como documentos JSON o información geográfica a través de la extensión PostGIS, y destaca por su excelente rendimiento en operaciones concurrentes masivas. Debido a su alto nivel de consistencia y flexibilidad, se utiliza de manera ideal en aplicaciones empresariales complejas, plataformas analíticas y sistemas de información geográfica.

MariaDB

Nació como una bifurcación directa de MySQL creada por sus desarrolladores originales con el objetivo de garantizar una alternativa que permaneciera completamente libre y de código abierto tras la adquisición de MySQL por parte de Oracle. Mantiene una compatibilidad total con su predecesor, funcionando como un

reemplazo directo en la gran mayoría de las infraestructuras, al mismo tiempo que introduce mejoras significativas de rendimiento y motores de almacenamiento optimizados. Es utilizado en servidores web basados en Linux y en servicios en la nube que requieren alta velocidad de procesamiento sin costos de licenciamiento.

MongoDB

Es el motor no relacional orientado a documentos más popular del mercado, caracterizado por almacenar la información en documentos flexibles en formato BSON, una variante binaria de JSON, en lugar de utilizar tablas y columnas tradicionales. Al no requerir esquemas rígidos predefinidos, permite adaptar y modificar la estructura de los datos dinámicamente a medida que evoluciona la aplicación. Su diseño está pensado para la escalabilidad horizontal mediante técnicas de distribución de datos entre múltiples servidores. Útil para el manejo de Big Data, catálogos de productos variables, desarrollo de aplicaciones móviles modernas y plataformas web con flujos de datos cambiantes.

Redis

Funciona como una base de datos no relacional de estructura de clave-valor que opera principalmente dentro de la memoria RAM del servidor, lo que le permite alcanzar velocidades de lectura y escritura medidas en microsegundos. Aunque prioriza el almacenamiento volátil para maximizar el rendimiento, dispone de mecanismos configurables para asegurar la persistencia de los datos en el disco duro. Admite una gran variedad de estructuras de datos como listas, hashes y conjuntos. Se emplea fundamentalmente para la implementación de sistemas de almacenamiento en caché, administración de sesiones en tiempo real, contadores de actividad rápida y colas de mensajería.

SQLite

Se diferencia de otros motores relacionales porque no opera bajo una arquitectura cliente-servidor ni corre como un servicio independiente en el sistema operativo. En su lugar, la base de datos completa se integra de forma embebida dentro de la propia aplicación y se almacena en un único archivo físico en el disco local. Es un motor muy ligero, no requiere configuración previa y consume una cantidad mínima de recursos de hardware. Sus casos de uso ideales se enfocan en el almacenamiento local en aplicaciones móviles para iOS y Android, software de escritorio, navegadores web y prototipos rápidos de desarrollo.

Cassandra

Desarrollada inicialmente por Facebook y transferida a la Fundación Apache, consiste en una base de datos NoSQL distribuida y orientada a columnas diseñada para gestionar volúmenes masivos de datos distribuidos en una gran cantidad de servidores. Su arquitectura en anillo no tiene un nodo maestro central, lo que significa que todos los puntos de la red operan con la misma jerarquía, asegurando una alta disponibilidad constante y una tolerancia a fallos buena donde la caída de un servidor no interrumpe

el servicio. Se utiliza para procesar flujos de datos gigantescos en tiempo real, registrar logs de actividad en redes sociales y almacenar telemetría de dispositivos de internet de las cosas.

Oracle Database

Es un sistema de gestión relacional multimodelo de carácter comercial y propietario que representa el estándar de rendimiento y seguridad dentro del sector corporativo global. Proporciona una robustez única para el procesamiento de transacciones críticas a gran escala, herramientas avanzadas de cifrado y opciones sofisticadas de alta disponibilidad diseñadas para evitar cualquier interrupción del negocio. Sus altos costos de licenciamiento restringen su uso a entornos donde la máxima fiabilidad es obligatoria, siendo implementado con frecuencia en la banca internacional, compañías aseguradoras de primer nivel y grandes sistemas de planificación de recursos empresariales.

SQL Server

Desarrollado por Microsoft, es un motor relacional de nivel empresarial diseñado para integrarse nativamente con todo el ecosistema de herramientas de la compañía, incluyendo entornos de desarrollo en .NET, la infraestructura en la nube de Azure y plataformas analíticas como Power BI. Ofrece herramientas integradas muy potentes dedicadas a la inteligencia de negocios, la integración y el análisis de datos complejos. Aunque estuvo ligado en exclusividad a los entornos Windows, en la actualidad cuenta con soporte para sistemas Linux y contenedores Docker, siendo elegido por corporaciones de tamaño mediano y grande para administrar almacenes de datos y aplicaciones empresariales críticas.

Tipos de Bases de Datos

Bases de datos Relacionales

Estas bases de datos se basan en el modelo relacional clásico, lo que significa que organizan la información en tablas estructuradas con filas y columnas estrictamente definidas. Cada fila representa un registro único y las tablas se conectan entre sí mediante claves conceptuales para evitar la duplicidad de datos y asegurar la consistencia. Entre sus ejemplos MySQL, PostgreSQL y Oracle. Sus usos principales se encuentran en sistemas donde la precisión y el cumplimiento de reglas transaccionales estrictas son obligatorios, tales como los sistemas de planificación de recursos empresariales, aplicaciones de facturación y plataformas de banca financiera.

Bases de datos Documentales

Pertenecientes a la familia NoSQL, almacenan la información en archivos independientes llamados documentos, que suelen utilizar un formato flexible idéntico o similar a JSON. Al no requerir un esquema fijo inicial, permiten que cada documento

guarde campos distintos y que la estructura evolucione dinámicamente con la aplicación. Su ejemplo principal es MongoDB. Se utilizan para el desarrollo de interfaces de programación de aplicaciones modernas que conectan servicios web, así como para gestionar catálogos de productos de comercio electrónico con atributos variables y sistemas de gestión de contenido dinámicos.

Bases de datos de Caché

Se enfoca en la velocidad extrema de respuesta, logrando su objetivo al procesar y mantener los datos temporalmente de forma nativa dentro de la memoria RAM del servidor en lugar de guardarlos directamente en el almacenamiento en disco tradicional. De ejemplo esta Redis. Su uso consiste en optimizar el rendimiento general de las aplicaciones web al almacenar consultas repetitivas de alta demanda para que el servidor no tenga que consultar la base de datos principal constantemente, acelerando drásticamente los tiempos de carga para el usuario final.

Bases de datos de Clave-Valor

Es un modelo NoSQL bastante simplificado donde cada elemento de información se guarda bajo una estructura de diccionario básica compuesta por una clave única que sirve como identificador directo y un valor asociado que contiene los datos, que pueden ser desde un texto simple hasta un objeto complejo. Redis es el ejemplo de este caso. Su uso está fuertemente enfocado en la gestión de sesiones de usuario en tiempo real en plataformas de alta concurrencia, contadores de actividad inmediata y sistemas de mensajería rápida donde la prioridad es recuperar un registro de manera instantánea conociendo su clave.

Bases de datos de Grafos

Están diseñadas específicamente para priorizar el análisis de las conexiones, estas bases de datos representan la información mediante nodos que actúan como las entidades de datos, propiedades que describen a esos nodos y aristas que definen explícitamente las relaciones directas que existen entre ellos. El ejemplo principal es Neo4j. Sus usos indispensables se centran en entornos que requieren mapear y recorrer interconexiones masivas en milisegundos, siendo la tecnología base para estructurar redes sociales, desarrollar sistemas de recomendación inteligente de contenido o compras, y diseñar plataformas complejas para la detección de fraudes mediante patrones de comportamiento vinculados.

Comparación SQL vs NoSQL

Esquema

Las bases de datos SQL utilizan un esquema fijo, lo que significa que la estructura de tablas, columnas y tipos de datos debe definirse antes de almacenar información. Esto permite mantener una organización estricta y ordenada de los datos. En cambio, las bases de datos NoSQL utilizan un esquema flexible, permitiendo almacenar distintos

tipos de información sin necesidad de seguir una estructura rígida, facilitando la adaptación a cambios constantes.

Relaciones

Las bases de datos SQL manejan relaciones fuertes entre los datos mediante claves primarias y foráneas. Esto permite conectar información de distintas tablas de forma segura y consistente. Por otro lado, las bases de datos NoSQL utilizan relaciones más flexibles, ya que muchas veces almacenan la información relacionada dentro del mismo documento o estructura, reduciendo la dependencia entre conjuntos de datos.

Escalabilidad

Las bases de datos SQL suelen escalar de forma vertical, aumentando la capacidad del servidor mediante más memoria, procesador o almacenamiento. Este enfoque es efectivo, aunque puede resultar costoso a medida que crecen las necesidades. En contraste, las bases de datos NoSQL están diseñadas para escalar horizontalmente, distribuyendo la carga entre múltiples servidores, lo que facilita el manejo de grandes volúmenes de información.

Consistencia

Las bases de datos SQL siguen las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), haciendo que las transacciones se ejecuten de manera segura y confiable. Esto resulta fundamental en sistemas donde la precisión de los datos es crítica. En cambio, muchas bases de datos NoSQL utilizan el modelo BASE (Basically Available, Soft State, Eventual Consistency), priorizando la disponibilidad y el rendimiento, aunque la consistencia de los datos pueda alcanzarse de forma gradual.

Ideal para

Las bases de datos SQL son ideales para aplicaciones que requieren transacciones complejas y una integridad de los datos, como sistemas bancarios, contables o de gestión empresarial. Sin embargo, las bases de datos NoSQL son más adecuadas para aplicaciones que manejan grandes volúmenes de información y necesitan alta escalabilidad, como redes sociales, plataformas de streaming o sistemas de análisis de datos masivos.

Casos de uso moderno

Kubernetes

Permite automatizar el despliegue, escalado y administración de aplicaciones en contenedores, facilitando la gestión de grandes infraestructuras.

Microservicios

Se utiliza para desarrollar aplicaciones divididas en servicios independientes que pueden actualizarse y escalarse por separado.

APIs

Permiten la comunicación entre diferentes aplicaciones y sistemas, facilitando el intercambio de datos y funcionalidades.

Cloud Native

Se emplea para crear aplicaciones diseñadas específicamente para aprovechar entornos de computación en la nube.

Big Data

Permite almacenar y procesar enormes cantidades de datos para obtener información útil y apoyar la toma de decisiones.

IA y ML

Se utiliza para desarrollar sistemas capaces de aprender de los datos, realizar predicciones y automatizar tareas inteligentes.

Tiempo real

Permite procesar y responder a eventos de forma inmediata, siendo fundamental en monitoreo, videojuegos y sistemas financieros.

IoT

Se utiliza para conectar dispositivos físicos a internet, permitiendo recopilar datos y controlarlos de forma remota.

Laboratorio

El laboratorio lo hice utilizando docker sobre la maquina virtual. Además, elegí utilizar mysql.

Parte 4 — Azure Cloud VM y Challenge 2

En esta parte se realiza la creación y configuración de una máquina virtual Ubuntu en Azure Cloud. El objetivo es comprender cómo desplegar infraestructura en la nube, configurar reglas de red y preparar el entorno para el uso de bases de datos y servicios.

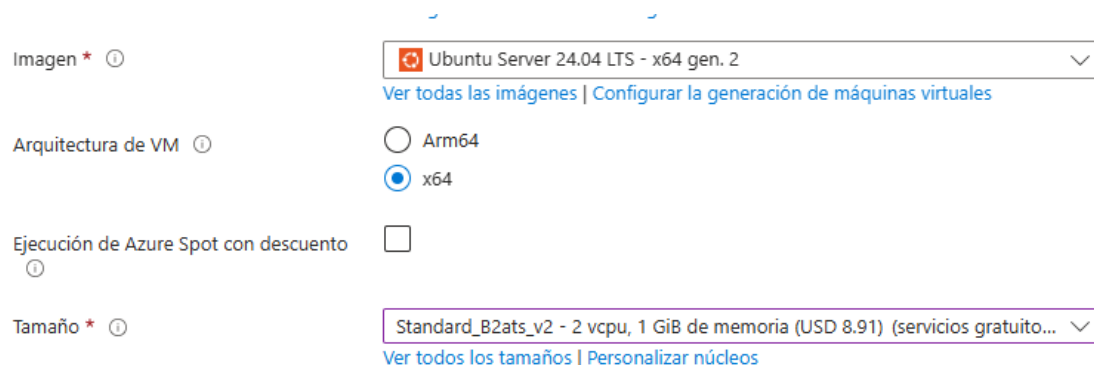


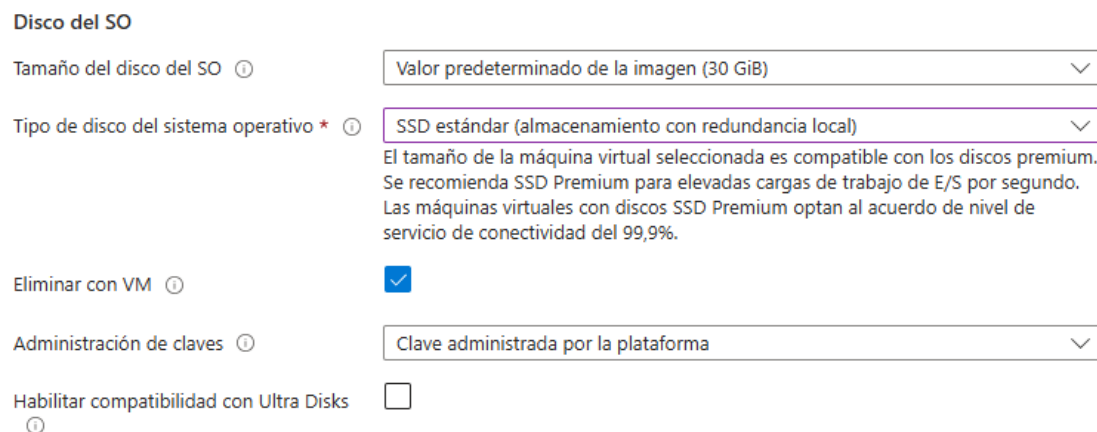
Imagen * ⓘ Ubuntu Server 24.04 LTS - x64 gen. 2
[Ver todas las imágenes](#) | [Configurar la generación de máquinas virtuales](#)

Arquitectura de VM ⓘ
☐ Arm64
☒ x64

Ejecución de Azure Spot con descuento ⓘ ☐

Tamaño * ⓘ Standard_B2ats_v2 - 2 vcpu, 1 GiB de memoria (USD 8.91) (servicios gratuito...)
[Ver todos los tamaños](#) | [Personalizar núcleos](#)

En esta captura se puede observar la creación de la máquina virtual en Azure, seleccionando Ubuntu 24.04 como sistema operativo y el tamaño de la instancia que será utilizada para el laboratorio.



Disco del SO

Tamaño del disco del SO ⓘ Valor predeterminado de la imagen (30 GiB)

Tipo de disco del sistema operativo * ⓘ SSD estándar (almacenamiento con redundancia local)
El tamaño de la máquina virtual seleccionada es compatible con los discos premium. Se recomienda SSD Premium para elevadas cargas de trabajo de E/S por segundo. Las máquinas virtuales con discos SSD Premium optan al acuerdo de nivel de servicio de conectividad del 99,9%.

Eliminar con VM ⓘ ☒

Administración de claves ⓘ Clave administrada por la plataforma

Habilitar compatibilidad con Ultra Disks ⓘ ☐

Aquí se selecciona un disco SSD estándar para almacenar el sistema operativo y los datos de la máquina virtual.



Región * ⓘ (Africa) South Africa North
[Implementación en una zona extendida de Azure](#)

En esta parte se configura la región donde será desplegada la máquina virtual. Se eligió la región de África debido a su disponibilidad y correcto funcionamiento.

Conectar
Iniciar
Reiniciar
Detener
Hibernar
Captura
Eliminar
Actualizar
Escalar

^ Información esencial

Grupo de recursos [\(mover\)](#)
[Lab4_group](#)

Estado
Detenido (desasignado)

Ubicación
South Africa North (Zona 1)

Suscripción [\(mover\)](#)
[Azure for Students](#)
Id. de suscripción

Sistema operativo
Linux

Tamaño
Standard_B2ats_v2

IP pública de NIC principal
[102.37.130.86](#)
[1 direcciones IP públicas asociadas](#)

Red virtual/subred
[Lab4-vnet/default](#)

La máquina virtual fue creada correctamente y ya se encuentra disponible para comenzar las configuraciones del laboratorio.

```
PS C:\Users\andre> ssh AndresR@102.37.130.86
AndresR@102.37.130.86's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1013-azure x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

En esta captura se muestra la conexión remota a la máquina virtual utilizando el usuario y contraseña configurados previamente.

Grupo de seguridad de red [Lab4-nsg](#) (conectado a networkinterface: [lab439_z1](#))
Afecta a 0 subredes, 1 interfaces de red

+ Crear ACL del puerto

Buscar reglas

Origen == todo
Destino == todo
Protocolo == todo
Acción == todo
Puerto == todo

Prio...	Nombre	Puerto	Protocolo	Origen	Destino	Acción
Reglas de puerto de entrada (4)						
300	SSH	22	TCP	Cualquiera	Cualquiera	Allow
65000	AllowVnetInBound	Cualquiera	Cualquiera	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerInB...	Cualquiera	Cualquiera	AzureLoadBalancer	Cualquiera	Allow
65500	DenyAllInBound	Cualquiera	Cualquiera	Cualquiera	Cualquiera	Deny
> Reglas de puerto de salida (3)						

Aquí se accede a la configuración de networking de Azure para administrar las reglas de acceso y los puertos de la máquina virtual.

Protocolo

- ☒ Any
☒ TCP
☐ UDP
☐ ICMPv4
☐ ICMPv6

Acción

- ☒ Permitir
☐ Denegar

Prioridad * ⓘ

310

Nombre *

mysql

Descripción

Origen ⓘ

Any

Intervalos de puertos de origen * ⓘ

*

Destino ⓘ

Any

Servicio ⓘ

Custom

Intervalos de puertos de destino * ⓘ

3306

En esta parte se configura la regla de red correspondiente a MySQL para permitir conexiones mediante el puerto 3306.

Intervalos de puertos de destino * ⓘ

5432



Protocolo

- ☐ Any
- ☒ TCP
- ☐ UDP
- ☐ ICMPv4
- ☐ ICMPv6

Acción

- ☒ Permitir
- ☐ Denegar

Prioridad * ⓘ

320

Nombre *

postgres



Aquí se agrega la configuración necesaria para permitir conexiones a PostgreSQL mediante el puerto 5432.

27017

Protocolo

- ☐ Any
- ☒ TCP
- ☐ UDP
- ☐ ICMPv4
- ☐ ICMPv6

Acción

- ☒ Permitir
- ☐ Denegar

Prioridad * ⓘ

330

Nombre

mongodb

En esta captura se habilita el puerto utilizado por MongoDB para permitir conexiones externas al servicio.

6379

Protocolo

☐ Any

☒ TCP

☐ UDP

☐ ICMPv4

☐ ICMPv6

Acción

☒ Permitir

☐ Denegar

Prioridad * ⓘ

340 ✓

Nombre

redis

Aquí se configura el puerto correspondiente a Redis para permitir el acceso al servicio desde la red.

Reglas de puerto de entrada (8)							
300	SSH		22	TCP	Cualquiera	Cualquiera	Allow
310	mysql		3306	TCP	Cualquiera	Cualquiera	Allow
320	postgres		5432	TCP	Cualquiera	Cualquiera	Allow
330	mongodb		27017	TCP	Cualquiera	Cualquiera	Allow
340	redis		6379	TCP	Cualquiera	Cualquiera	Allow

Todas las reglas de red necesarias fueron configuradas correctamente en Azure.

```
AndresR@Lab4:~$ sudo ufw allow 3306/tcp
Rules updated
Rules updated (v6)
AndresR@Lab4:~$ sudo ufw allow 5432/tcp
Rules updated
Rules updated (v6)
AndresR@Lab4:~$ sudo ufw allow 27017/tcp
Rules updated
Rules updated (v6)
AndresR@Lab4:~$ sudo ufw allow 6379/tcp
Rules updated
Rules updated (v6)
```

Además de las reglas de Azure, también se realizaron configuraciones adicionales utilizando permisos de administrador en Ubuntu.

```
AndresR@Lab4:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
AndresR@Lab4:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

Finalmente, se habilita el puerto 22 para conexiones SSH y se activa el firewall para mejorar la seguridad del servidor.

Máquina virtual		Redes	
Nombre del equipo	Lab4	Dirección IP pública	102.37.130.86 (Interfaz de red lab439_z1)
Sistema operativo	Linux (ubuntu 24.04)		1 direcciones IP públicas asociadas
Generación de VM	V2	Dirección IP pública (IPv6)	-
Arquitectura de VM	x64	Dirección IP privada	10.3.0.4
Estado del agente	Ready	Dirección IP privada (IPv6)	-
Versión del agente	2.15.2.1		

Acá se ven la ip pública y la ip privada para el challenge 2.

Parte 2. Instalación de Bases de Datos Relacionales

En esta parte se realiza la instalación y configuración inicial de MySQL en Ubuntu. Además, se verifican los servicios, se habilita el inicio automático y se aplican medidas básicas de seguridad.

```
AndresR@Lab4:~$ sudo apt update && sudo apt upgrade -y
Get:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 Packages [1401 kB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/main Translation-en [513 kB]
```

```
Restarting services...
systemctl restart chrony.service packagekit.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
AndresR@Lab4:~$
```

En esta captura se actualizan los paquetes y repositorios de Ubuntu para asegurar que el sistema se encuentre al día.

```
AndresR@Lab4:~$ sudo apt install mysql-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Aquí se realiza la instalación del servidor MySQL utilizando el gestor de paquetes de Ubuntu.

```
AndresR@Lab4:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-05-21 21:02:35 UTC; 1min 59s ago
     Process: 7450 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
    Main PID: 7459 (mysqld)
      Status: "Server is operational"
        Tasks: 37 (limit: 988)
      Memory: 363.3M (peak: 378.9M)
         CPU: 1.263s
       CGroup: /system.slice/mysql.service
              └─7459 /usr/sbin/mysqld

May 21 21:02:33 Lab4 systemd[1]: Starting mysql.service - MySQL Community Server...
May 21 21:02:35 Lab4 systemd[1]: Started mysql.service - MySQL Community Server.
```

En esta parte se verifica que el servicio de MySQL se encuentre funcionando correctamente.

```
AndresR@Lab4:~$ sudo systemctl enable mysql
Synchronizing state of mysql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable mysql
```

Aquí se configura MySQL para que inicie automáticamente junto con el sistema operativo.

```
AndresR@Lab4:~$ sudo mysql_secure_installation

Securing the MySQL server deployment.

Connecting to MySQL using a blank password.

VALIDATE PASSWORD COMPONENT can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?

Press y|Y for Yes, any other key for No: n

Skipping password set for root as authentication with auth_socket is used by default.
If you would like to use password authentication instead, this can be done with the "ALTER_USER" command.
See https://dev.mysql.com/doc/refman/8.0/en/alter-user.html#alter-user-password-management for more information.
```

En esta captura comienza el proceso de configuración segura de MySQL, donde se definen parámetros de seguridad y contraseñas.

```
By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.

Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.
```

Aquí se eliminan los usuarios anónimos de MySQL para evitar accesos no autorizados.

```
Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : y
Success.
```

En esta parte se deshabilita el acceso remoto del usuario root para mejorar la seguridad del servidor.

```
By default, MySQL comes with a database named 'test' that anyone can access. This is also intended only for testing, and should be removed before moving into a production environment.
```

```
Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y
- Dropping test database...
Success.

- Removing privileges on test database...
Success.
```

Aquí se aplican configuraciones adicionales relacionadas con la base de datos y la seguridad general de MySQL.

```
Reloading the privilege tables will ensure that all changes made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
```

En esta captura se actualizan y aplican los privilegios de los usuarios configurados en MySQL.

```
AndresR@Lab4:~$ sudo mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

Aquí se accede al cliente CLI de MySQL desde la terminal para administrar la base de datos mediante comandos.

La sección restante de la Parte 2 al elegir MySQL entre este y PostgreSQL, no se realizará en esta parte ni en las siguientes. Por letra del laboratorio.

Parte 3. Administración Linux y Servicios, y Challenge 1

En esta parte se utilizan herramientas de administración y monitoreo de Linux para controlar los servicios de bases de datos, verificar procesos activos y observar el uso de recursos del sistema.

```

AndresR@Lab4:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-05-21 21:02:35 UTC; 55min ago
     Main PID: 7459 (mysqld)
    Status: "Server is operational"
      Tasks: 39 (limit: 988)
     Memory: 404.8M (peak: 407.6M)
        CPU: 15.920s
    CGroup: /system.slice/mysql.service
           └─7459 /usr/sbin/mysqld

May 21 21:02:33 Lab4 systemd[1]: Starting mysql.service - MySQL Community Server...
May 21 21:02:35 Lab4 systemd[1]: Started mysql.service - MySQL Community Server.

```

En esta captura se utiliza systemctl para verificar el estado actual del servicio MySQL.

```

AndresR@Lab4:~$ sudo systemctl stop mysql

```

```

AndresR@Lab4:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: inactive (dead) since Thu 2026-05-21 21:59:48 UTC; 44s ago
     Duration: 57min 12.344s
   Process: 7459 ExecStart=/usr/sbin/mysqld (code=exited, status=0/SUCCESS)
    Main PID: 7459 (code=exited, status=0/SUCCESS)
    Status: "Server shutdown complete"
        CPU: 16.543s

May 21 21:02:33 Lab4 systemd[1]: Starting mysql.service - MySQL Community Server...
May 21 21:02:35 Lab4 systemd[1]: Started mysql.service - MySQL Community Server.
May 21 21:59:47 Lab4 systemd[1]: Stopping mysql.service - MySQL Community Server...
May 21 21:59:48 Lab4 systemd[1]: mysql.service: Deactivated successfully.
May 21 21:59:48 Lab4 systemd[1]: Stopped mysql.service - MySQL Community Server.
May 21 21:59:48 Lab4 systemd[1]: mysql.service: Consumed 16.543s CPU time, 407.6M memory peak, 0B memory swap peak.

```

Aquí se ejecuta el comando para detener el servicio MySQL y luego se verifica que efectivamente quedó detenido.

```

AndresR@Lab4:~$ sudo systemctl start mysql

```

En esta parte se inicia nuevamente el servicio MySQL desde la terminal.

```

AndresR@Lab4:~$ systemctl restart mysql
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to restart 'mysql.service'.
Authenticating as: Ubuntu (AndresR)
Password:
==== AUTHENTICATION COMPLETE ====

```

Aquí se reinicia el servicio MySQL utilizando permisos de administrador, por lo que el sistema solicita la contraseña.

```

AndresR@Lab4:~$ ps aux | grep mysql
mysql      8131  1.2 45.6 1786408 395620 ?        Ssl  22:04   0:00 /usr/sbin/mysqld
AndresR    8183  0.0  0.2   7084   2144 pts/0    S+   22:05   0:00 grep --color=auto mysql

```

En esta captura se muestran los procesos activos relacionados con MySQL dentro del sistema.


```

AndresR@Lab4:~$ top
top - 22:36:53 up 1:49, 1 user, load average: 0.07, 0.02, 0.00
Tasks: 129 total, 1 running, 128 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.3 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 846.9 total, 67.8 free, 762.6 used, 159.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 84.3 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 7818 root        20   0     0     0     0   I   0.3   0.0   0:00.34 kworker/1:0-events
 8131 mysql      20   0 1786408 385300 9380  S   0.3  44.4   0:09.78 mysqld
    1 root        20   0    22816 11928  7484  S   0.0   1.4   0:04.37 systemd
    2 root        20   0     0     0     0   S   0.0   0.0   0:00.00 kthreadd
    3 root        20   0     0     0     0   S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root        0 -20     0     0     0   I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root        0 -20     0     0     0   I   0.0   0.0   0:00.00 kworker/R-sync_wq

```

Aquí se utiliza el comando top para monitorear en tiempo real el uso de recursos y procesos del sistema.

```

0[          0.0%] Tasks: 30, 62 thr, 100 kthr; 1 running
1[          0.7%] Load average: 0.00 0.00 0.00
Mem[|||||623M/847M] Uptime: 01:51:41
Swp[          0K/0K]

Main  I/O
  PID USER      PRI  NI   VIRT   RES   SHR  S  CPU% MEM%    TIME+  Command
 8146 mysql      20   0 1744M  376M  9380  S   0.0 44.4   0:00.56 /usr/sbin/mysqld
    1 root        20   0    22816 11928  7484  S   0.0  1.4   0:04.38 /sbin/init
 132 root        19  -1 66896 13828 12624  S   0.0  1.6   0:00.72 /usr/lib/systemd/systemd-journald
 195 root        RT   0    282M  27324  8760  S   0.0  3.2   0:00.14 /sbin/multipathd -d -s
 197 root        20   0    25792  5524  2904  S   0.0  0.6   0:00.13 /usr/lib/systemd/systemd-udev
 200 root        20   0    282M  27324  8760  S   0.0  3.2   0:00.00 /sbin/multipathd -d -s
 202 root        RT   0    282M  27324  8760  S   0.0  3.2   0:00.00 /sbin/multipathd -d -s
 203 root        RT   0    282M  27324  8760  S   0.0  3.2   0:00.00 /sbin/multipathd -d -s
 204 root        RT   0    282M  27324  8760  S   0.0  3.2   0:00.00 /sbin/multipathd -d -s
 205 root        RT   0    282M  27324  8760  S   0.0  3.2   0:00.32 /sbin/multipathd -d -s
 206 root        RT   0    282M  27324  8760  S   0.0  3.2   0:00.00 /sbin/multipathd -d -s
 496 systemd-re 20   0    21592  7024  4692  S   0.0  0.8   0:00.14 /usr/lib/systemd/systemd-resolved
 632 systemd-ne 20   0    19140  5344  4008  S   0.0  0.6   0:00.05 /usr/lib/systemd/systemd-networkd
 677 messagebus  20   0    9908  5184  4252  S   0.0  0.6   0:00.47 @dbus-daemon --system --address=systemd: --nofork --n
 682 root        20   0    32428 13460  3216  S   0.0  1.6   0:00.09 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-s
 685 polkitd    20   0    306M  8436  5992  S   0.0  1.0   0:00.08 /usr/lib/polkit-1/polkitd --no-debug
 694 root        20   0    18212  6872  5776  S   0.0  0.8   0:00.16 /usr/lib/systemd/systemd-logind
 696 root        20   0    458M  7864  5616  S   0.0  0.9   0:00.06 /usr/libexec/udisks2/udisksd
 697 root        20   0    43980 27928  4252  S   0.0  3.2   0:00.42 /usr/bin/python3 -u /usr/sbin/waagent -daemon
 701 syslog     20   0    217M  5292  3620  S   0.0  0.6   0:00.03 /usr/sbin/rsyslogd -n -iNONE
 713 root        20   0    7232  2364  2108  S   0.0  0.3   0:00.01 /usr/sbin/cron -f -P
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit

```

En esta parte se utiliza htop, una herramienta más visual e interactiva para monitorear procesos y recursos.

```

AndresR@Lab4:~$ free -m
              total          used          free      shared  buff/cache   available
Mem:           846          763           65           3         161          83
Swap:            0            0            0

```

Aquí se utiliza el comando free -m para visualizar el uso de memoria RAM del sistema.

```

AndresR@Lab4:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root       29G   2.7G   26G  10% /
tmpfs           424M    0  424M   0% /dev/shm
tmpfs          170M  988K  169M   1% /run
tmpfs           5.0M    0   5.0M   0% /run/lock
efivarfs       128K   38K   86K  31% /sys/firmware/efi/efivars
/dev/sda16     881M  117M  703M  15% /boot
/dev/sda15     105M   6.2M   99M   6% /boot/efi
tmpfs          85M   12K   85M   1% /run/user/1000

```

En esta captura se utiliza el comando df para visualizar el uso y espacio disponible en los discos del sistema.

Parte 5. Docker y Contenedores, y Challenge 3

En esta parte se realiza la instalación y configuración de Docker dentro de Ubuntu. Además, se crean distintos contenedores de bases de datos y se verifica su funcionamiento mediante comandos, logs y administración de servicios.

```
Last login: Thu May 21 22:36:42 2026 from 190.133.6.139
AndresR@Lab4:~$ sudo apt update && sudo apt install -y docker.io
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
```

En esta captura se observa la instalación de Docker dentro de Ubuntu. Docker permitirá ejecutar contenedores aislados que contienen servicios completos, como bases de datos y aplicaciones, sin necesidad de instalar todo directamente en el sistema operativo.

```
AndresR@Lab4:~$ sudo systemctl enable docker
AndresR@Lab4:~$ sudo systemctl start docker
```

Aquí se habilita el servicio de Docker y se configura para iniciarse automáticamente junto con Ubuntu. Esto permite que los contenedores puedan utilizarse incluso después de reiniciar la máquina virtual.

```
AndresR@Lab4:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

En esta parte se verifica que Docker esté funcionando correctamente utilizando comandos administrativos. Se utiliza sudo debido a que Docker necesita permisos elevados para interactuar con los servicios y contenedores del sistema.

```
AndresR@Lab4:~$ sudo docker run -d \
--name mysql-lab \
-e MYSQL_ROOT_PASSWORD=admin123 \
-p 3306:3306 \
mysql:8
Unable to find image 'mysql:8' locally
8: Pulling from library/mysql
b9f6fc0eea0c: Pull complete
ab549d4e00e7: Pull complete
1dced004aaec: Pull complete
f148eeb1a828: Pull complete
d3bae0286aad: Pull complete
63b1a26b5169: Pull complete
```

Durante esta etapa fue necesario aumentar los recursos de la máquina virtual porque la configuración inicial no tenía suficiente memoria o capacidad para descargar y ejecutar correctamente los contenedores. Luego de aumentar el tamaño de la VM, la instalación y descarga funcionaron correctamente.

```
AndresR@Lab4:~$ sudo docker run -d \
--name mongodb-lab \
-p 27017:27017 \
mongo:7
Unable to find image 'mongo:7' locally
7: Pulling from library/mongo
a436614bf543: Pull complete
3611ecf4f0d7: Pull complete
40d16f30db40: Pull complete
e2b4fd11adff: Pull complete
```

Aquí se descarga y ejecuta el contenedor correspondiente a MongoDB utilizando Docker. El contenedor queda funcionando de manera independiente dentro de la máquina virtual.

```
AndresR@Lab4:~$ sudo docker run -d \
--name redis-lab \
-p 6379:6379 \
redis:7
Unable to find image 'redis:7' locally
7: Pulling from library/redis
0251943f3cb8: Download complete
```

En esta captura se ejecuta el contenedor de Redis. Este servicio funcionará como una base de datos en memoria enfocada en velocidad y almacenamiento temporal.

```
AndresR@Lab4:~$ sudo docker run -d \
--name postgres-lab \
-e POSTGRES_PASSWORD=admin123 \
-p 5432:5432 \
postgres:16
Unable to find image 'postgres:16' locally
16: Pulling from library/postgres
57b072481e62: Download complete
```

Aquí también se crea un contenedor de PostgreSQL para disponer de otra base de datos relacional adicional dentro del laboratorio.

```
AndresR@Lab4:~$ sudo docker logs postgres-lab
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
```

En esta captura se visualizan los logs del contenedor PostgreSQL. Los mensajes indican el inicio correcto del servicio, la carga de configuraciones internas y que el servidor quedó preparado para aceptar conexiones desde clientes externos.

```
AndresR@Lab4:~$ sudo systemctl stop mysql
AndresR@Lab4:~$ sudo docker start mysql-lab
mysql-lab
AndresR@Lab4:~$ sudo docker logs mysql-lab
2026-05-28 20:27:28+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9 started.
2026-05-28 20:27:28+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-05-28 20:27:28+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9 started.
2026-05-28 20:27:28+00:00 [Note] [Entrypoint]: Initializing database files
2026-05-28T20:27:28.506072Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2026-05-28T20:27:28.507636Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.9) initializing
ogress as process 80
```

Para evitar conflictos de puertos, fue necesario detener primero la instalación local de MySQL realizada directamente en Ubuntu. Luego se inició el contenedor Docker para que pudiera utilizar correctamente el puerto 3306.

```
AndresR@Lab4:~$ sudo docker ps
CONTAINER ID   IMAGE      NAMES      COMMAND      CREATED      STATUS      PORTS
f6f1add6baa9   postgres:16   postgres-lab   "docker-entrypoint.s..."   7 minutes ago   Up 7 minutes   0.0.0.0:5432->5432/tcp, [:::]
5432->5432/tcp
58b6530da714   redis:7       redis-lab     "docker-entrypoint.s..."   7 minutes ago   Up 7 minutes   0.0.0.0:6379->6379/tcp, [:::]
6379->6379/tcp
57e7225c7b81   mongo:7       mongodb-lab   "docker-entrypoint.s..."   10 minutes ago   Up 10 minutes   0.0.0.0:27017->27017/tcp, [:::]
27017->27017/tcp
530e43f21e1a   mysql:8       mysql-lab     "docker-entrypoint.s..."   15 minutes ago   Up About a minute   3306->3306/tcp, [:::]
3306->3306/tcp
```

En esta captura se puede observar que los contenedores fueron creados correctamente y se encuentran ejecutándose junto con los puertos asignados a cada servicio. El puerto de MySQL no aparece inicialmente debido al conflicto previo con la instalación local.

Como parte del challenge 3, las siguientes respuestas:

Diferencia entre VM y contenedor

Una máquina virtual virtualiza todo un sistema operativo completo, incluyendo su propio kernel, lo que consume más recursos. En cambio, un contenedor comparte el kernel del sistema anfitrión y solo incluye las dependencias necesarias para ejecutar una aplicación, por lo que es más liviano y rápido.

Qué proceso mantiene vivo el contenedor

Un contenedor se mantiene en ejecución mientras el proceso principal que fue definido al iniciarlo siga activo. Si ese proceso termina o falla, el contenedor también se detiene automáticamente.

Ventajas de Docker

- Utiliza menos recursos que una máquina virtual.
- Permite desplegar aplicaciones de forma rápida y sencilla.
- Facilita la portabilidad entre distintos entornos.
- Aísla aplicaciones y dependencias para evitar conflictos.
- Simplifica la administración y escalabilidad de servicios.

Parte 6. SQL básico y Challenge 4

En esta parte se realizan operaciones SQL básicas utilizando MySQL ejecutado en Docker. El objetivo es practicar comandos CRUD y verificar cómo se almacenan y modifican los datos dentro de una base de datos relacional.

```
AndresR@Lab4:~$ sudo docker exec -it mysql-lab mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.9 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input s

mysql> |
```

Aquí se accede al contenedor de MySQL utilizando comandos Docker desde la máquina virtual. Esto permite administrar la base de datos directamente desde la terminal.

```
mysql> CREATE DATABASE ejemplo;
Query OK, 1 row affected (0.01 sec)
```

En esta captura se crea una nueva base de datos utilizando el comando CREATE DATABASE dentro de MySQL.

```
mysql> USE ejemplo;
Database changed
```

Aquí se selecciona la base de datos creada mediante el comando USE para comenzar a trabajar sobre ella.

```
mysql> CREATE TABLE usuarios (
->   id INT PRIMARY KEY,
->   nombre VARCHAR(50),
->   email VARCHAR(100)
-> );
Query OK, 0 rows affected (0.04 sec)
```

En esta parte se crea una tabla definiendo las columnas, tipos de datos y claves necesarias para almacenar información.

```
mysql> INSERT INTO usuarios VALUES
    -> (1, 'Juan', 'juan@test.com'),
    -> (2, 'Maria', 'maria@test.com'),
    -> (3, 'Pedro', 'pedro@test.com');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Aquí se insertan registros dentro de la tabla utilizando instrucciones INSERT INTO.

```
mysql> SELECT * FROM usuarios;
+----+-----+-----+
| id | nombre | email          |
+----+-----+-----+
|  1 | Juan   | juan@test.com  |
|  2 | Maria  | maria@test.com |
|  3 | Pedro  | pedro@test.com |
+----+-----+-----+
3 rows in set (0.00 sec)
```

En esta captura se ejecuta una consulta SELECT para verificar que la tabla fue creada correctamente y que los registros fueron almacenados sin errores.

```
mysql> UPDATE usuarios
    -> SET nombre='Juan Carlos'
    -> WHERE id=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Aquí se modifica uno de los registros de la tabla utilizando el comando UPDATE.

```
mysql> SELECT * FROM usuarios;
+----+-----+-----+
| id | nombre      | email          |
+----+-----+-----+
|  1 | Juan Carlos | juan@test.com  |
|  2 | Maria       | maria@test.com |
|  3 | Pedro       | pedro@test.com |
+----+-----+-----+
3 rows in set (0.00 sec)
```

En esta parte se realiza otra consulta para confirmar que el cambio realizado anteriormente fue aplicado correctamente sobre el registro.

```
mysql> DELETE FROM usuarios
    -> WHERE id=3;
Query OK, 1 row affected (0.01 sec)
```

Aquí se elimina uno de los registros de la tabla utilizando el comando DELETE.

```
mysql> SELECT * FROM usuarios;
```

id	nombre	email
1	Juan Carlos	juan@test.com
2	Maria	maria@test.com

```
2 rows in set (0.00 sec)
```

Verifico, y veo que el usuario 3 fue eliminado. Con esto completo la parte 3 y el challenge de sql al usar create, update, insert, select y delete.

Parte 7. Mongo db y Challenge 5

En esta parte se utiliza MongoDB para trabajar con bases de datos NoSQL orientadas a documentos. Se realizan operaciones CRUD utilizando documentos flexibles almacenados dentro de colecciones.

```
AndresR@Lab4:~$ sudo docker exec -it mongodb-lab mongosh
Current Mongosh Log ID: 6a18aa5bbfeb3010529df8a2
Connecting to:      mongodb://127.0.0.1:27017/?directConnection
&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      7.0.34
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and
to MongoDB periodically (https://www.mongodb.com/legal/privacy-pol
You can opt-out by running the disableTelemetry() command.
```

Aquí se accede al contenedor MongoDB utilizando la terminal interactiva mongosh para comenzar a administrar la base de datos.

```
test> use laboratorio
switched to db laboratorio
```

En esta captura se crea y selecciona la base de datos llamada laboratorio dentro de MongoDB.

```
laboratorio> db.usuarios.insertOne({
|   nombre: "Juan",
|   edad: 25,
|   ciudad: "Montevideo"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a18aae5bfeb3010529df8a3')
}
```

Aquí se inserta un documento dentro de una colección utilizando una estructura flexible basada en JSON.

```
laboratorio> db.usuarios.find()
[
  {
    _id: ObjectId('6a18aae5bfeb3010529df8a3'),
    nombre: 'Juan',
    edad: 25,
    ciudad: 'Montevideo'
  }
]
```

En esta parte se consulta la colección para visualizar el documento almacenado dentro de MongoDB.

```
laboratorio> db.usuarios.find({
|   edad: { $gt: 20 }
| })
[
  {
    _id: ObjectId('6a18aae5bfeb3010529df8a3'),
    nombre: 'Juan',
    edad: 25,
    ciudad: 'Montevideo'
  }
]
```

Aquí se realiza una búsqueda utilizando filtros para obtener únicamente los documentos que cumplen determinadas condiciones.


```
laboratorio> db.usuarios.updateOne(
|   { nombre: "Juan" },
|   {
|     $set: {
|       ciudad: "Maldonado"
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

En esta captura se modifica información existente dentro de un documento utilizando updateOne.

```
laboratorio> db.usuarios.find()
[
  {
    _id: ObjectId('6a18aae5bfeb3010529df8a3'),
    nombre: 'Juan',
    edad: 25,
    ciudad: 'Maldonado'
  }
]
```

Aquí se verifica que el documento haya sido actualizado correctamente dentro de la colección.

```
laboratorio> db.usuarios.deleteOne({
|   nombre: "Juan"
| })
{ acknowledged: true, deletedCount: 1 }
```

En esta parte se elimina un documento de la colección utilizando deleteOne.

```
laboratorio> db.usuarios.find()

laboratorio>
```

Finalmente, se realiza una consulta adicional para confirmar que el documento fue eliminado correctamente.

Parte 8. Redis, base de datos cache y challenge 6

En esta parte se trabaja con Redis, una base de datos NoSQL de tipo clave-valor orientada al almacenamiento en memoria. Se realizan operaciones sobre claves, contadores, sesiones temporales, listas y hashes para comprender las estructuras de datos más utilizadas en Redis.

```
AndresR@Lab4:~$ sudo docker start redis-lab
redis-lab
AndresR@Lab4:~$ sudo docker exec -it redis-lab redis-cli
127.0.0.1:6379> |
```

En esta captura se inicia Redis y se accede a su consola interactiva mediante redis-cli. Desde este entorno es posible ejecutar comandos directamente sobre la base de datos y observar los resultados en tiempo real.

```
127.0.0.1:6379> SET usuario "Juan"
OK
```

Aquí se crea una clave dentro de Redis utilizando una operación básica de almacenamiento. Las claves funcionan como identificadores únicos que permiten acceder rápidamente a la información guardada.

```
127.0.0.1:6379> GET usuario
"Juan"
```

En esta parte se recupera el valor asociado a la clave creada en la anterior captura para verificar que los datos fueron almacenados correctamente.

Luego cree una clave que será utilizada como contador. Este tipo de estructura sirve para registrar visitas, eventos o estadísticas dentro de aplicaciones. Me olvide de tomarle captura, pero el comando fue este: SET visitas 0.

```
127.0.0.1:6379> INCR visitas
(integer) 1
```

En esta captura se incrementa automáticamente el valor del contador.

```
127.0.0.1:6379> SET sesion "activa"
OK
```

Aquí se crea una sesión temporal dentro de Redis. Este mecanismo es utilizado para almacenar información de usuarios conectados en aplicaciones web.

```
127.0.0.1:6379> EXPIRE sesion 60
(integer) 1
```

En esta parte se configura un tiempo de expiración de 60 segundos para la sesión creada. Una vez alcanzado ese tiempo, Redis eliminará automáticamente la clave.

```
127.0.0.1:6379> TTL sesion
(integer) 48
```

Esta captura muestra la consulta del tiempo restante antes de que la clave expire. Se puede observar cuántos segundos faltan para que Redis elimine la información.

```
127.0.0.1:6379> LPUSH tareas "backup"  
(integer) 1  
127.0.0.1:6379> LPUSH tareas "deploy"  
(integer) 2
```

Aquí se crean listas dentro de Redis. Las listas permiten almacenar múltiples elementos manteniendo un orden específico de inserción.

```
127.0.0.1:6379> LRange tareas 0 -1  
1) "deploy"  
2) "backup"
```

En esta parte se consulta el contenido de las listas para comprobar que los elementos fueron almacenados correctamente.

```
127.0.0.1:6379> HSET usuario:1 nombre "Juan"  
(integer) 1  
127.0.0.1:6379> HSET usuario:1 edad 25  
(integer) 1
```

Aquí se crea un hash, una estructura que permite almacenar múltiples pares campo-valor dentro de una misma clave, similar a un objeto o registro.

```
127.0.0.1:6379> HGETALL usuario:1  
1) "nombre"  
2) "Juan"  
3) "edad"  
4) "25"
```

En esta captura se visualiza el contenido completo del hash para verificar que todos los datos fueron guardados correctamente.

Parte 9. Seguridad y Usuarios

En esta sección se administran usuarios y permisos dentro de MySQL. El objetivo es aprender a crear cuentas independientes, asignar privilegios específicos y aplicar medidas básicas de seguridad para controlar el acceso a las bases de datos.

```
AndresR@Lab4:~$ sudo docker start mysql-lab
mysql-lab
AndresR@Lab4:~$ sudo docker exec -it mysql-lab mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.9 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement

mysql> |
```

Aquí se accede al servidor MySQL desde la terminal para realizar tareas de administración relacionadas con usuarios y permisos.

```
mysql> USE ejemplo;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

En esta parte se selecciona la base de datos sobre la cual se aplicarán las configuraciones de seguridad y control de acceso.

```
mysql> CREATE USER 'appuser'@'%' IDENTIFIED BY 'Password123!';
Query OK, 0 rows affected (0.05 sec)
```

Aquí se crea un nuevo usuario dentro de MySQL, permitiendo disponer de credenciales independientes para acceder a la base de datos.

```
mysql> GRANT ALL PRIVILEGES ON ejemplo.* TO 'appuser'@'%';
Query OK, 0 rows affected (0.00 sec)
```

En esta captura se asignan privilegios específicos al usuario creado, definiendo qué acciones podrá realizar sobre la base de datos.

```
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)
```

Aquí se actualizan y aplican los permisos configurados para que MySQL reconozca inmediatamente las modificaciones realizadas.

```
mysql> SELECT User, Host FROM mysql.user;
```

User	Host
appuser	%
root	%
mysql.infoschema	localhost
mysql.session	localhost
mysql.sys	localhost
root	localhost

```
6 rows in set (0.00 sec)
```

En esta parte se consulta la lista de usuarios para confirmar que la cuenta fue creada correctamente y que los permisos fueron aplicados.

Parte 10. Backup y Restore

En esta parte se realizan tareas de respaldo y recuperación de información. Se genera una copia de seguridad de una base de datos, se simula una pérdida de datos y finalmente se restaura el contenido para comprobar la efectividad del backup.

```
AndresR@Lab4:~$ sudo docker exec mysql-lab mysqldump -u root -padmin123 ejemplo > backup.sql
mysqldump: [Warning] Using a password on the command line interface can be insecure.
```

En esta captura se genera una copia de seguridad de la base de datos utilizando las herramientas de respaldo de MySQL. El archivo generado permite recuperar la información en caso de pérdida.

```
AndresR@Lab4:~$ ls -l backup.sql
-rw-rw-r-- 1 AndresR AndresR 1989 Jun  8 22:40 backup.sql
```

Aquí se verifica que el archivo de respaldo fue creado correctamente y que se encuentra disponible para futuras restauraciones.

```
mysql> DROP DATABASE ejemplo;
Query OK, 1 row affected (0.05 sec)

mysql> SHOW DATABASES;
```

Database
information_schema
mysql
performance_schema
sys

```
4 rows in set (0.00 sec)
```

En esta parte se realiza una simulación de pérdida de información eliminando o modificando datos de prueba para comprobar la utilidad del respaldo.

```
AndresR@Lab4:~$ sudo docker exec -i mysql-lab mysql -u root -padmin123 ejemplo < backup.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
```

Aquí se utiliza el archivo de respaldo para restaurar el contenido original de la base de datos.

```
mysql> USE ejemplo;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker prompt

Database changed
mysql> SELECT * FROM usuarios;
+----+-----+-----+
| id | nombre   | email          |
+----+-----+-----+
| 1  | Juan Carlos | juan@test.com  |
| 2  | Maria      | maria@test.com |
+----+-----+-----+
2 rows in set (0.00 sec)
```

En esta captura se verifica que los datos (de partes anteriores) fueron recuperados correctamente después del proceso de restauración.

Parte 11 Automatización Bash

En esta sección se desarrolla un script Bash para automatizar la creación de copias de seguridad. El script genera automáticamente un backup de la base de datos MySQL, almacenando la información en un archivo para facilitar su recuperación ante pérdidas de datos o fallos del sistema.

```
GNU nano 7.2 backup.sh *
#!/bin/bash

DATE=$(date +%F)

docker exec mysql-lab mysqldump -u root -padmin123 ejemplo > backup-$DATE.sql

echo "Backup completado"
```

Aquí se desarrolla un script Bash encargado de automatizar la creación de copias de seguridad. El script reúne todos los comandos necesarios para generar respaldos de forma rápida y repetible.

```
AndresR@Lab4:~$ nano backup.sh
AndresR@Lab4:~$ chmod +x backup.sh
```

En esta parte se asignan permisos de ejecución al script mediante chmod, permitiendo que pueda ejecutarse directamente desde la terminal.

```
AndresR@Lab4:~$ sudo ./backup.sh
mysqldump: [Warning] Using a password on the command line interface can be insecure.
Backup completado
```

Aquí se ejecuta el script y se observa el mensaje de confirmación indicando que el proceso de respaldo finalizó correctamente.

```
AndresR@Lab4:~$ ls
backup-2026-06-08.sql  backup.sh  backup.sql
```

Finalmente, se comprueba que el archivo de respaldo fue generado correctamente y que la automatización funcionó según lo esperado.

Challenge 7

El challenge 7 pide crear una estructura Github para el contenido del laboratorio.

```
PS C:\Users\andre\UCU\Sistemas> mkdir Laboratorio5-Challenge7

Directorio: C:\Users\andre\UCU\Sistemas

Mode                LastWriteTime         Length Name
----                -
d-----          9/6/2026   9:59 a. m.          Laboratorio5-Challenge7
```

Creo la carpeta principal.

```
PS C:\Users\andre\UCU\Sistemas> cd Laboratorio5-Challenge7
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> |
```

Me muevo a la carpeta creada.

```
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> mkdir scripts
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> mkdir docker
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> mkdir sql
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> mkdir evidencias
```

Creo las carpetas que exige el challenge.

```
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> New-Item README.md -ItemType File
```

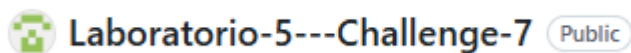
Creo el archivo readme.md vacío.

```
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> dir

Directorio: C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7

Mode                LastWriteTime         Length Name
----                -
d-----          9/6/2026  10:05 a. m.         docker
d-----          9/6/2026  10:05 a. m.         evidencias
d-----          9/6/2026  10:04 a. m.         scripts
d-----          9/6/2026  10:04 a. m.         sql
-a-----          9/6/2026  10:09 a. m.           0 README.md
```

Verifico que se creó correctamente.



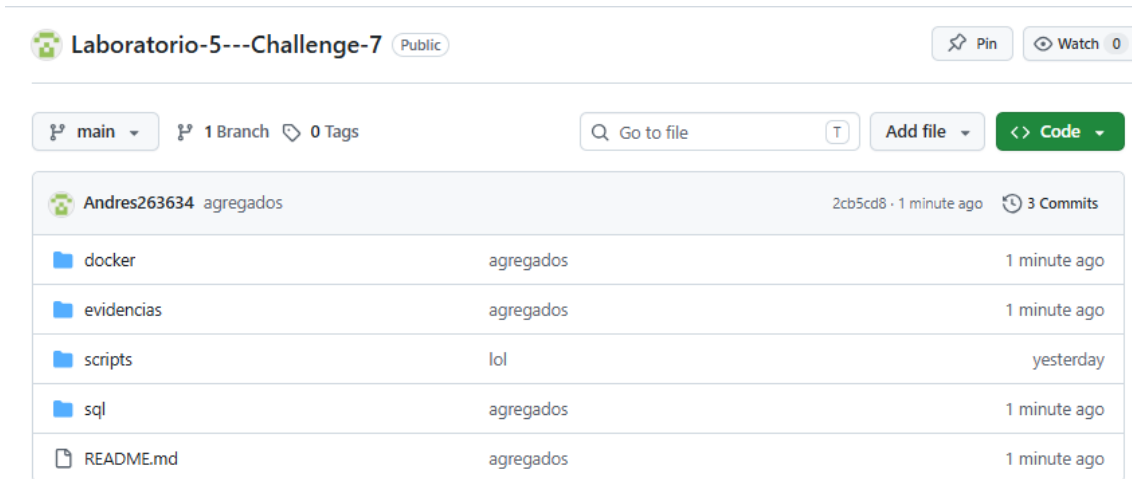
Creo el repositorio en GitHub

```
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> git init
Initialized empty Git repository in C:/Users/andre/UCU/Sistemas/Laboratorio5-Challenge7/.git/
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> git add .
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> git commit -m "Agrego todos los cambios que hice"
[master (root-commit) 168b125] Agrego todos los cambios que hice
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
```

Inicializo el git en la carpeta, agrego los cambios y hago commit.

```
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> git remote add origin https://github.com/Andres263634/Laboratorio5---Challenge-7
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> git branch -M main
PS C:\Users\andre\UCU\Sistemas\Laboratorio5-Challenge7> git push -u origin main
Enumerating objects: 3, done.
```

Agrego los cambios al repositorio en GitHub.



Finalmente se ve así. Con la parte de docker, evidencias, scripts, sql, readme, y en un momento el informe.

Problemas encontrados

El principal problema que encontré fue que, al hacer la máquina virtual, la hice con un tamaño aparentemente muy chico para este laboratorio. Use un tamaño de B2 y lo cambie por uno D4. Después, la instalación y el uso de maquina virtual fue mas fluido y sin trabas.

Conclusión

Diferencias SQL vs NoSQL

Las bases de datos SQL utilizan una estructura relacional con esquemas definidos y son ideales para aplicaciones que requieren integridad y consistencia de los datos. Por otro lado, las bases de datos NoSQL ofrecen una estructura más flexible y permiten almacenar información sin un esquema fijo, siendo adecuadas para aplicaciones con grandes volúmenes de datos o requisitos de escalabilidad.

Diferencias VM vs Docker

Las máquinas virtuales ejecutan un sistema operativo completo sobre un entorno virtualizado, lo que implica un mayor consumo de recursos. Docker utiliza contenedores que comparten el kernel del sistema anfitrión, permitiendo una ejecución más ligera, rápida y eficiente de las aplicaciones.

Importancia de Redis

Redis es una base de datos en memoria utilizada principalmente como sistema de caché. Su alta velocidad permite reducir los tiempos de respuesta de las aplicaciones, mejorar el rendimiento y disminuir la carga sobre las bases de datos principales.

Impacto del Cloud Computing

El cloud computing permite acceder a recursos informáticos bajo demanda sin necesidad de adquirir infraestructura física propia. Esto facilita la escalabilidad, reduce costos operativos y permite desplegar servicios de forma rápida y eficiente desde cualquier ubicación.

Seguridad y Backups

La seguridad es fundamental para proteger la información almacenada en las bases de datos. La creación de usuarios con permisos adecuados, la configuración de puertos y firewalls, y la realización periódica de copias de seguridad permiten reducir riesgos y garantizar la recuperación de los datos ante fallos o incidentes.